

飞腾基础线性代数库用户指南

V1.4 (2025-03-20)

版权所有 © 飞腾信息技术有限公司 2022。保留所有权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium和其他飞腾商标均为飞腾信息技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

本文档的内容视为飞腾的保密信息，您应当严格遵守保密义务；未经飞腾事先书面同意，您不得向任何第三方披露本文档内容或提供给任何第三方使用。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，飞腾在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但飞腾在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

本文档中所有内容，包括但不限于图片、架构设计、页面布局、文字描述，均由飞腾和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权等。非经飞腾和/或其关联公司书面同意，任何人不得擅自使用、修改、复制上述内容。
如有技术问题，可联系 support@phytium.com.cn 获取支持。

飞腾信息技术有限公司

地址：天津市滨海新区海缘路1号信安创业广场5号楼

网址：www.phytium.com.cn

版本历史

文档版本	发布日期	更新说明
V1.0	2024-03-22	首次发布
V1.1	2024-06-25	版本更新
V1.2	2024-09-28	版本更新
V1.3	2024-12-20	版本更新
V1.4	2025-03-20	版本更新

目 录

1 概述	6
2 安装指南	7
2.1 环境要求	7
2.1.1 硬件环境	7
2.1.2 软件环境	7
2.2 BLAS-FT 函数库安装和使用	7
2.2.1 部署 BLAS-FT 函数库	7
2.2.2 使用 BLAS-FT 函数库	8
2.2.3 验证 BLAS-FT 函数库	8
3 BLAS-FT 函数说明	9
3.1 函数说明	9
3.2 数据结构	9
4 函数定义	10
4.1 BLAS-FT Level 1 函数	10
4.1.1 函数说明	10
4.1.2 ?asum	10
4.1.3 ?axpy	11
4.1.4 ?copy	11
4.1.5 ?dot	12
4.1.6 ?sdot	12
4.1.7 ?dotc	13
4.1.8 ?dotu	14
4.1.9 ?nrm2	14
4.1.10 ?rot	15
4.1.11 ?rotg	15

4.1.12 ?rotm	16
4.1.13 ?rotmg	16
4.1.14 ?scal	17
4.1.15 ?swap	18
4.1.16 i?amax	18
4.1.17 i?amin	19
4.2 BLAS-FT Level 2 函数	19
4.2.1 函数说明	19
4.2.2 ?gbmv	20
4.2.3 ?gemv	22
4.2.4 ?ger	23
4.2.5 ?gerc	24
4.2.6 ?geru	25
4.2.7 ?hbmvm	26
4.2.8 ?hemv	27
4.2.9 ?her	27
4.2.10 ?her2	28
4.2.11 ?hpmv	29
4.2.12 ?hpr	30
4.2.13 ?hpr2	31
4.2.14 ?sbmv	31
4.2.15 ?spmv	32
4.2.16 ?spr	33
4.2.17 ?spr2	34
4.2.18 ?symv	35
4.2.19 ?syr	35
4.2.20 ?syr2	36
4.2.21 ?tbmv	37
4.2.22 ?tbsv	38
4.2.23 ?tpmv	39
4.2.24 ?tpsv	41
4.2.25 ?trmv	42
4.2.26 ?trsv	43
4.3 BLAS-FT Level 3 函数	44

4.3.1 函数说明	44
4.3.2 ?gemm	44
4.3.3 ?hemm	46
4.3.4 ?herk	47
4.3.5 ?her2k	48
4.3.6 ?symm	50
4.3.7 ?syrk	51
4.3.8 ?syr2k	52
4.3.9 ?trmm	54
4.3.10 ?trsm	56

表 目 录

表 2.1 软件环境	7
------------------	---

1 概述

飞腾基础线性代数库 BLAS-FT (Basic Linear Algebra Subprograms for Phytium) 是一组定义了基本线性代数运算的接口和函数集合,包括向量-向量运算,向量-矩阵运算,矩阵-矩阵运算。BLAS-FT 基于飞腾处理器体系结构特点和指令集特点进行了优化,实现了在飞腾平台上的性能提升。

2 安装指南

2.1 环境要求

2.1.1 硬件环境

飞腾基础线性代数库紧密支持飞腾处理器，已在 FT-2000/4、飞腾腾锐 D2000、FT-2000+/64、飞腾腾云 S2500、飞腾腾云 S5000C 处理器上成熟运用。

2.1.2 软件环境

软件环境如表 2.1 所示。

表 2.1 软件环境

项目	版本
OS	centos8.2、Kylin V10、 Ubuntu19.10、openEuler 22.03
GCC	9.2.1、7.3.0、8.3.1、9.3.0、 10.3.1
GNU Binutils	2.32、2.33、2.34、2.35、 2.37
Glibc	2.28、2.23、2.31、2.30、 2.34

2.2 BLAS-FT 函数库安装和使用

2.2.1 部署 BLAS-FT 函数库

步骤一 获取并解压 BLAS-FT.zip，解压缩目录下包含 BLAS-FT 的库文件以及头文件。

步骤二 设置库路径，生成的库在解压文件夹中的 lib 目录下。

```
export LD_LIBRARY_PATH=/home/BLAS-FT/lib:$LD_LIBRARY_PATH
```

步骤三 执行 source 命令或重新登陆终端让环境变量生效。

```
source /etc/profile
```

步骤四 查看环境变量 LD_LIBRARY_PATH 是否包含 BLAS-FT 的安装路径“/home/BLAS-FT/lib”。

```
env | grep LD_LIBRARY_PATH
```

如果包含安装路径则安装成功。

2.2.2 使用 BLAS-FT 函数库

请在 gcc 编译选项中添加动态库所在路径，链接需要使用的动态库文件，添加编译选项后用 ldd 指令检查程序依赖库是否准确链接。

```
-L./lib -I./include -lblas_ft
```

2.2.3 验证 BLAS-FT 函数库

步骤一 获取并解压 BLAS-FT.zip，解压目录下包含 BLAS-FT 的库文件、头文件以及测试文件。

步骤二 进入 test/ctest 目录中。

```
cd test && cd ctest
```

步骤三 编译目录

```
make clean && make -j USE_OPENMP=1 TARGET=FT2000
```

步骤四 若没有错误，则显示 END OF TESTS，验证完毕。

3

BLAS-FT 函数说明

3.1 函数说明

BLAS-FT 库中的函数可以根据其实现的功能和计算复杂度分为三个级别：

1、BLAS Level 1：包括对单个向量进行操作的函数，如点积、向量范数、向量加法等。

2、BLAS Level 2：包括对矩阵与向量之间进行操作的函数，如矩阵-向量乘法以及解线性方程组等。

3、BLAS Level 3：包括对矩阵与矩阵之间进行操作的函数，最典型的是矩阵乘法。

3.2 数据结构

数据类型	类型描述
s	单精度浮点类型
d	双精度浮点类型
c	单精度复数类型
z	双精度复数类型

4 函数定义

4.1 BLAS-FT Level 1 函数

4.1.1 函数说明

函数族	数据类型	描述
?asum	s、d、sc、dz	向量元素的绝对值之和
?axpy	s、d、c、z	向量与标量的乘积，并将结果加到另一个向量中
?copy	s、d、c、z	将一个向量复制到另一个向量
?dot	s、d	向量与向量的点积
?sdot	s、d	双精度的向量与向量的点积
?dotc	c、z	共轭向量与另一个向量的点积
?dotu	c、z	两个复数向量的点积
?nrm2	s、d、sc、dz	向量的欧几里得范数
?rot	s、d、sc、dz	在平面上执行平面旋转
?rotg	s、d	计算吉文斯旋转平面
?rotm	s、d	修改的吉文斯旋转平面
?rotmg	s、d	修改的吉文斯旋转参数
?scal	s、d、c、z、cs、dz	向量缩放
?swap	s、d、c、z	向量互换
i?amax	s、d、c、z	找到具有最大绝对值的元素的索引
i?amin	s、d、c、z	找到具有最小绝对值的元素的索引

4.1.2 ?asum

向量元素的绝对值之和。

接口定义

```
float cblas_sasum(const BLASINT n, const float *x, const BLASINT incx);
```

```
double cblas_dasum(const BLASINT n, const double *x, const BLASINT incx);
```

```
float cblas_scasum(const BLASINT n, const void *x, const BLASINT
```

```
incx);
```

```
double cblas_dzasum(const BLASINT n, const void *x, const BLASINT  
incx);
```

输入参数

n	指定向量 x 中的元素数量
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量

输出参数

res	包含向量所有元素实部和虚部之和
-----	-----------------

4.1.3 ?axpy

向量与标量的乘积，并将结果加到另一个向量中。

接口定义

```
void cblas_saxpy(const BLASINT n, const float alpha, const float *x,  
const BLASINT incx, float *y, const BLASINT incy);
```

```
void cblas_daxpy(const BLASINT n, const double alpha, const double  
*x, const BLASINT incx, double *y, const BLASINT incy);
```

```
void cblas_caxpy(const BLASINT n, const void *alpha, const void *x,  
const BLASINT incx, void *y, const BLASINT incy);
```

```
void cblas_zaxpy(const BLASINT n, const void *alpha, const void *x,  
const BLASINT incx, void *y, const BLASINT incy);
```

输入参数

n	指定向量 x 和 y 中的元素数量
alpha	系数标量
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y，至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

y	更新后的 y
---	--------

4.1.4 ?copy

将一个向量复制到另一个向量。

接口定义

```
void cblas_scopy(const BLASINT n, const float *x, const BLASINT incx,
```

```
float *y, const BLASINT incy);  
void cblas_dcopy(const BLASINT n, const double *x, const BLASINT  
incx, double *y, const BLASINT incy);  
void cblas_ccopy(const BLASINT n, const void *x, const BLASINT incx,  
void *y, const BLASINT incy);  
void cblas_zcopy(const BLASINT n, const void *x, const BLASINT incx,  
void *y, const BLASINT incy);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

y	如果 n 为正, 则包含向量 x 的副本, 否则, 参数不变
---	--------------------------------

4.1.5 ?dot

向量与向量的点积。

接口定义

```
float cblas_sdot(const BLASINT n, const float *x, const BLASINT incx,  
const float *y, const BLASINT incy);  
double cblas_ddot(const BLASINT n, const double *x, const BLASINT  
incx, const double *y, const BLASINT incy);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

res	如果 n 为正, 则返回 x 和 y 点积结果, 否则返回 0
-----	---------------------------------

4.1.6 ?sdot

双精度的向量与向量的点积。

接口定义

```
float cblas_sdsdot(const BLASINT n, const float alpha, const float *x,  
const BLASINT incx, const float *y, const BLASINT incy);
```

```
double cblas_dsdot(const BLASINT n, const float *x, const BLASINT  
incx, const float *y, const BLASINT incy);
```

输入参数

n	指定向量 x 和 y 中的元素数量
alpha	单精度标量，加到点积中返回
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y，至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

res	如果 n 为正,则为 x 和 y 的点积结果(对于 sdsdot 还需加上 alpha)，否则对于 sdsdot, res 包含 alpha, 对于 dsdot, res 包含 0
-----	--

4.1.7 ?dotc

共轭向量与另一个向量的点积。

接口定义

```
float _Complex cblas_cdotc(const BLASINT n, const void *x, const  
BLASINT incx, const void *y, const BLASINT incy);
```

```
void cblas_cdotc_sub(const BLASINT n, const void *x, const BLASINT  
incx, const void *y, const BLASINT incy, void *ret);
```

```
double _Complex cblas_zdotc(const BLASINT n, const void *x, const  
BLASINT incx, const void *y, const BLASINT incy);
```

```
void cblas_zdotc_sub(const BLASINT n, const void *x, const BLASINT  
incx, const void *y, const BLASINT incy, void *ret);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y，至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

dotc 如果 n 为正, 它为共轭 x 和非共轭 y 点积的结果。
否则, 它为 0。

4.1.8 ?dotu

两个复数向量的点积。

接口定义

```
float _Complex cblas_cdotu(const BLASINT n, const void *x, const
BLASINT incx, const void *y, const BLASINT incy);
```

```
void cblas_cdotu_sub(const BLASINT n, const void *x, const BLASINT
incx, const void *y, const BLASINT incy, void *ret);
```

```
double _Complex cblas_zdotu(const BLASINT n, const void *x, const
BLASINT incx, const void *y, const BLASINT incy);
```

```
void cblas_zdotu_sub(const BLASINT n, const void *x, const BLASINT
incx, const void *y, const BLASINT incy, void *ret);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x , 至少为 $(1+(n-1)*abs(incx))$
$incx$	向量 x 索引的增量
y	向量 y , 至少为 $(1+(n-1)*abs(incy))$
$incy$	向量 y 索引的增量

输出参数

dotu	如果 n 为正, 则返回 x 和 y 点积结果, 否则返回 0
------	---------------------------------------

4.1.9 ?nrm2

向量的欧几里得范数。

接口定义

```
float cblas_snrm2 (const BLASINT n, const float *x, const BLASINT
incx);
```

```
double cblas_dnrm2 (const BLASINT n, const double *x, const
BLASINT incx);
```

```
float cblas_scnrm2 (const BLASINT n, const float *x, const BLASINT
incx);
```

```
double cblas_dznrm2 (const BLASINT n, const double *x, const
BLASINT incx);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量

输出参数

x	向量 x 的欧几里得范数
---	--------------

4.1.10 ?rot

在平面上执行平面旋转。

接口定义

```
void cblas_srot(const BLASINT n, float *x, const BLASINT incx, float *y,
const BLASINT incy, const float c, const float s);
```

```
void cblas_drot(const BLASINT n, double *x, const BLASINT incX,
double *y, const BLASINT incy, const double c, const double s);
```

```
void cblas_csrot(const BLASINT n, float *x, const BLASINT incx, float *y,
const BLASINT incy, const float c, const float s);
```

```
void cblas_zdrot(const BLASINT n, double *x, const BLASINT incx,
double *y, const BLASINT incy, const double c, const double s);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量
c	一个标量, 旋转角的余弦
s	一个标量, 旋转角的正弦

输出参数

dotu	如果 n 为正, 则返回 x 和 y 点积结果, 否则返回 0
x	x 中每个元素都被 $cx+sy$ 替换
y	y 中每个元素都被 $cy-sx$ 替换

4.1.11 ?rotg

吉文斯旋转平面。

接口定义

```
void cblas_srotg(float *a, float *b, float *c, float *s);
```

```
void cblas_drotg(double *a, double *b, double *c, double *s);
```

输入参数

a	x 坐标
b	y 坐标

输出参数

a	x 坐标
b	y 坐标
c	吉文斯旋转参数 c
s	吉文斯旋转参数 s

4.1.12 ?rotm

修改的吉文斯旋转平面。

接口定义

```
void cblas_srotm(const BLASINT n, float *x, const BLASINT incx, float *y, const BLASINT incy, const float *p);
```

```
void cblas_drotm(const BLASINT n, double *x, const BLASINT incx, double *y, const BLASINT incy, const double *p);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量
p	数组, 大小为 5, 参数数组的元素分别是: p[0]包含一个开关标志, p[1-4]分别包含数组 H 的组件 h_{11} 、 h_{21} 、 h_{12} 和 h_{22}

输出参数

x	每个元素 $x[i]$ 都会被 $h_{11}x[i]+h_{12}y[i]$ 替代
y	每个元素 $y[i]$ 都会被 $h_{21}x[i]+h_{22}y[i]$ 替代

4.1.13 ?rotmg

修改的吉文斯旋转参数。

接口定义

```
void cblas_srotmg(float *d1, float *d2, float *x, const float y, float *p);
```

```
void cblas_drotmg(double *d1, double *d2, double *x, const double y, double *p);
```

输入参数

d_1	输入向量 x 坐标的缩放因子
d_2	输入向量 y 坐标的缩放因子
x	输入向量 x 坐标
y	输入向量 y 坐标
输出参数	
d_1	更新后矩阵的第一个对角线元素
d_2	更新后矩阵的第二个对角线元素
x	旋转向量在缩放前的坐标 x
p	数组，大小为 5，参数数组的元素分别是： $p[0]$ 包含一个开关标志， $p[1-4]$ 分别包含数组 H 的组件 h_{11} 、 h_{21} 、 h_{12} 和 h_{22}

4.1.14 ?scal

向量缩放。

接口定义

```
void cblas_dscal(const BLASINT n, const double alpha, double *n,
const BLASINT incx);
```

```
void cblas_sscal(const BLASINT n, const float alpha, float *n, const
BLASINT incx);
```

```
void cblas_cscal(const BLASINT n, const void *alpha, void *n, const
BLASINT incx);
```

```
void cblas_zscal(const BLASINT n, const void *alpha, void *n, const
BLASINT incx);
```

```
void cblas_csscal(const BLASINT n, const double alpha, void *n, const
BLASINT incx);
```

```
void cblas_zdscal(const BLASINT n, const double alpha, void *n, const
BLASINT incx);
```

输入参数

n	指定向量 x 中的元素数量
α	系数标量
x	向量 x ，至少为 $(1+(n-1)*abs(incx))$
$incx$	向量 x 索引的增量

输出参数

x	更新后的 x
-----	----------

4.1.15 ?swap

向量互换。

接口定义

```
void cblas_sswap(const BLASINT n, float *x, const BLASINT incx, float *y, const BLASINT incy);
```

```
void cblas_dswap(const BLASINT n, double *x, const BLASINT incx, double *y, const BLASINT incy);
```

```
void cblas_cswap(const BLASINT n, void *x, const BLASINT incx, void *y, const BLASINT incy);
```

```
void cblas_zswap(const BLASINT n, void *x, const BLASINT incx, void *y, const BLASINT incy);
```

输入参数

n	指定向量 x 和 y 中的元素数量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 索引的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 索引的增量

输出参数

x	结果向量 x, 即输入向量 y
y	结果向量 y, 即输入向量 x

4.1.16 i?amax

找到具有最大绝对值的元素的索引。

接口定义

```
CBLAS_INDEX cblas_isamax(const BLASINT n, const float *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_idamax(const BLASINT n, const double *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_icamax(const BLASINT n, const void *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_izamax(const BLASINT n, const void *x, const BLASINT incx);
```

输入参数

n	指定向量 x 中的元素数量
---	---------------

x 向量 x，至少为 $(1+(n-1)*abs(incx))$

incx 向量 x 索引的增量

输出参数

index 实数为绝对值最大值索引，复数为实部绝对值与虚部绝对值之和最大值索引

4.1.17 i?amin

找到具有最小绝对值的元素的索引。

接口定义

```
CBLAS_INDEX cblas_isamin(const BLASINT n, const float *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_idamin(const BLASINT n, const double *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_icamin(const BLASINT n, const void *x, const BLASINT incx);
```

```
CBLAS_INDEX cblas_izamin(const BLASINT n, const void *x, const BLASINT incx);
```

输入参数

n 指定向量 x 中的元素数量

x 向量 x，至少为 $(1+(n-1)*abs(incx))$

incx 向量 x 索引的增量

输出参数

index 实数为绝对值最小值索引，复数为实部绝对值与虚部绝对值之和最小值索引

4.2 BLAS-FT Level 2 函数

4.2.1 函数说明

函数族	数据类型	描述
?gbmv	s、d、c、z	带状矩阵与向量的乘积
?gemv	s、d、c、z	一般矩阵与向量的乘积
?ger	s、d	一般矩阵秩 1 更新
?gerc	c、z	一般矩阵秩 1 更新（共轭）
?geru	c、z	一般矩阵秩 1 更新（非共轭）
?hbmV	c、z	厄米特带状矩阵与向量的乘积
?hemv	c、z	厄米特矩阵与向量的乘积

?her	c、z	厄米特矩阵秩 1 更新
?her2	c、z	厄米特矩阵秩 2 更新
?hpmv	c、z	厄米特压缩矩阵与向量的乘积
?hpr	c、z	厄米特压缩矩阵秩 1 更新
?hpr2	c、z	厄米特压缩矩阵秩 2 更新
?sbmv	s、d	对称带状矩阵与向量的乘积
?spmv	s、d	对称压缩矩阵与向量的乘积
?spr	s、d	对称压缩矩阵秩 1 更新
?spr2	s、d	对称压缩矩阵秩 2 更新
?symv	s、d	对称矩阵与向量的乘积
?syr	s、d	对称矩阵秩 1 更新
?syr2	s、d	对称矩阵秩 2 更新
?tbmv	s、d、c、z	三角带状矩阵与向量的乘积
?tbsv	s、d、c、z	求解系数为三角带状矩阵的线性方程组
?tpmv	s、d、c、z	三角压缩矩阵与向量的乘积
?tpsv	s、d、c、z	求解系数为三角压缩矩阵的线性方程组
?trmv	s、d、c、z	三角矩阵与向量的乘积
?trsv	s、d、c、z	求解系数为三角矩阵的线性方程组

4.2.2 ?gbmv

带状矩阵与向量的乘积。

① $y = \alpha * A * x + \beta * y$

② $y = \alpha * A' * x + \beta * y$

③ $y = \alpha * \text{conjg}(A') * x + \beta * y$

接口定义

```
void cblas_sgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transa, const BLASINT m, const BLASINT n, const BLASINT kl, const BLASINT ku, const float alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);
```

```
void cblas_dgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT kl, const BLASINT ku, const double alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx, const double beta,
```

```
double *y, const BLASINT incy);
```

```
void cblas_cgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT kl, const BLASINT ku, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

```
void cblas_zgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT kl, const BLASINT ku, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
m	矩阵 A 的行数, 大于等于 0
n	矩阵 A 的列数, 大于等于 0
kl	矩阵 A 的次对角阶数, 大于等于 0
ku	矩阵 A 的超对角阶数, 大于等于 0
alpha	系数标量
a	带状矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 的值必须至少为(kl+ku+1)
x	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 (1+(n-1)*abs(incx)) ; 否则至少为 (1+(m-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 (1+(n-1)*abs(incy)) ; 否则至少为 (1+(m-1)*abs(incy))
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.3 ?gemv

一般矩阵与向量的乘积。

① $y = \alpha * A * x + \beta * y$

② $y = \alpha * A' * x + \beta * y$

③ $y = \alpha * \text{conjg}(A') * x + \beta * y$

接口定义

```
void cblas_sgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const float alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);
```

```
void cblas_dgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const double alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx, const double beta, double *y, const BLASINT incy);
```

```
void cblas_cgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

```
void cblas_zgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
m	矩阵 A 的行数，大于等于 0
n	矩阵 A 的列数，大于等于 0
alpha	系数标量
a	一般矩阵 A，列存/行存，规模 lda*n/lda*m
lda	矩阵 A 的主维度，列存/行存，至少为 max(1,m)/max(1,n)

x	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 $(1+(n-1)*abs(incx))$; 否则至少为 $(1+(m-1)*abs(incx))$
incx	向量 x 的增量
beta	系数标量
y	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 $(1+(n-1)*abs(incy))$; 否则至少为 $(1+(m-1)*abs(incy))$
incy	向量 y 的增量
输出参数	
y	更新后的向量 y

4.2.4 ?ger

一般矩阵秩 1 更新。

$$A = \alpha * x * y' + A$$

接口定义

```
void cblas_sger(const enum CBLAS_ORDER order, const BLASINT m,
const BLASINT n, const float alpha, const float *x, const BLASINT incx,
const float *y, const BLASINT incy, float *a, const BLASINT lda);
```

```
void cblas_dger(const enum CBLAS_ORDER order, const BLASINT m,
const BLASINT n, const double alpha, const double *x, const BLASINT incx,
const double *y, const BLASINT incy, double *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
m	矩阵 A 的行数, 大于等于 0
n	矩阵 A 的列数, 大于等于 0
alpha	系数标量
x	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 $(1+(n-1)*abs(incx))$; 否则至少为 $(1+(m-1)*abs(incx))$
incx	向量 x 的增量
beta	系数标量
y	向量, 当 trans=CblasNoTrans 时, 向量大小至少为 $(1+(n-1)*abs(incy))$; 否则至少为 $(1+(m-1)*abs(incy))$

	$(1+(m-1)*abs(incy))$
incy	向量 y 的增量
a	一般矩阵 A，规模是 lda*n
lda	矩阵 A 的主维度，当 order 为 CblasColMajor 和 CblasRowMajor，lda 值必须至少为 max(1,m)和 max(1,n)
输出参数	
a	更新后的矩阵 A

4.2.5 ?gerc

一般矩阵秩 1 更新（共轭）。

$$A = \alpha * x * \text{conjg}(y') + A$$

接口定义

```
void cblas_cgerc(const enum CBLAS_ORDER order, const BLASINT
m, const BLASINT n, const void *alpha, const void *x, const BLASINT incx,
const void *y, const BLASINT incy, void *a, const BLASINT lda);
```

```
void cblas_zgerc(const enum CBLAS_ORDER order, const BLASINT
m, const BLASINT n, const void *alpha, const void *x, const BLASINT incx,
const void *y, const BLASINT incy, void *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序（CblasRowMajor）还是按列主序（CblasColMajor）
m	矩阵 A 的行数，大于等于 0
n	矩阵 A 的列数，大于等于 0
alpha	系数标量
x	向量，当 trans=CblasNoTrans 时，向量大小至少为 $(1+(n-1)*abs(incx))$ ；否则至少为 $(1+(m-1)*abs(incx))$
incx	向量 x 的增量
beta	系数标量
y	向量，当 trans=CblasNoTrans 时，向量大小至少为 $(1+(n-1)*abs(incy))$ ；否则至少为 $(1+(m-1)*abs(incy))$
incy	向量 y 的增量
a	一般矩阵 A，规模是 lda*n

lda	矩阵 A 的主维度，当 order 为 CblasColMajor 和 CblasRowMajor, lda 值必须至少为 max(1, m)和 max(1,n)
输出参数	
a	更新后的矩阵 A

4.2.6 ?geru

一般矩阵秩 1 更新（非共轭）。

$A = \alpha * x * y' + A$

接口定义

```
void cblas_cgeru(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *a, const BLASINT lda);  
void cblas_zgeru(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序（CblasRowMajor）还是按列主序（CblasColMajor）
m	矩阵 A 的行数，大于等于 0
n	矩阵 A 的列数，大于等于 0
alpha	系数标量
x	向量，当 trans=CblasNoTrans 时，向量大小至少为 (1+(n-1)*abs(incx))；否则至少为 (1+(m-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量，当 trans=CblasNoTrans 时，向量大小至少为 (1+(n-1)*abs(incy))；否则至少为 (1+(m-1)*abs(incy))
incy	向量 y 的增量
a	一般矩阵 A，规模是 lda*n
lda	矩阵 A 的主维度，当 order 为 CblasColMajor 和 CblasRowMajor，lda 值必须至少为 max(1,m)和 max(1,n)

输出参数

a

更新后的矩阵 A

4.2.7 ?hbmvm

厄米特带状矩阵与向量的乘积。

$$y = \alpha * A * x + \beta * y$$

接口定义

```
void cblas_chbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

```
void cblas_zhbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
k	矩阵 A 的超对角阶数, 大于等于 0
alpha	系数标量
a	厄米特带状矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为(k+1)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量 y, 至少为(1+(n-1)*abs(incy))
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.8 ?hemv

厄米特矩阵与向量的乘积。

$$y = \alpha * A * x + \beta * y$$

接口定义

```
void cblas_chemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

```
void cblas_zhemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
k	矩阵 A 的超对角阶数, 大于等于 0
alpha	系数标量
a	厄米特矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为 max(1,n)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量 y, 至少为(1+(n-1)*abs(incy))
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.9 ?her

厄米特矩阵秩 1 更新。

$$A = \alpha * x * \text{conjg}(x') + A$$

接口定义

```
void cblas_cher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const void *x, const BLASINT incx, void *a, const BLASINT lda);
```

```
void cblas_zher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const void *x, const BLASINT incx, void *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
a	厄米特矩阵 A, 规模是 $lda*n$
lda	矩阵 A 的主维度, lda 值必须至少为 $\max(1,n)$

输出参数

a	更新后的上三角/下三角矩阵 A
---	-----------------

4.2.10 ?her2

厄米特矩阵秩 2 更新。

$$A = \alpha * x * \text{conjg}(y') + \text{conjg}(\alpha) * y * \text{conjg}(x') + A$$
接口定义

```
void cblas_cher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *a, const BLASINT lda);
```

```
void cblas_zher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
a	厄米特矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为 $\max(1,n)$
输出参数	
a	更新后的上三角/下三角矩阵 A

4.2.11 ?hpmv

厄米特压缩矩阵与向量的乘积。

$y = \alpha * A * x + \beta * y$

接口定义

```
void cblas_chpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *ap, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

```
void cblas_zhpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *ap, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量

ap	压缩数组，规模至少是 $(n*(n+1)/2)$
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
beta	系数标量
y	向量 y，至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 的增量
输出参数	
y	更新后的向量 y

4.2.12 ?hpr

厄米特压缩矩阵秩 1 更新。

$A = \alpha * x * \text{conjg}(x') + A$

接口定义

```
void cblas_chpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const void *x, const BLASINT incx, void *a);
```

```
void cblas_zhpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const void *x, const BLASINT incx, void *a);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower，则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
n	矩阵 A 的阶数，大于等于 0
alpha	系数标量
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
ap	压缩数组，规模至少是 $(n*(n+1)/2)$

输出参数

ap	更新后的数组 ap
----	-----------

4.2.13 ?hpr2

厄米特压缩矩阵秩 2 更新。

$$A = \alpha * x * \text{conjg}(y') + \text{conjg}(\alpha) * y * \text{conjg}(x') + A$$

接口定义

```
void cblas_chpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ap);
```

```
void cblas_zhpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ap);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
x	向量 x, 至少为 $(1+(n-1)*\text{abs}(\text{incx}))$
incx	向量 x 的增量
y	向量 y, 至少为 $(1+(n-1)*\text{abs}(\text{incy}))$
incy	向量 y 的增量
ap	压缩数组, 规模至少是 $(n*(n+1)/2)$

输出参数

ap	更新后的数组 ap
----	-----------

4.2.14 ?sbmv

对称带状矩阵与向量的乘积。

$$A = \alpha * A * x + \beta * y$$

接口定义

```
void cblas_ssbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const BLASINT k, const float alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const
```

```
float beta, float *y, const BLASINT incy);
```

```
void cblas_dsbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const BLASINT k, const double alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx, const double beta, double *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
k	矩阵 A 的超对角阶数, 大于等于 0
alpha	系数标量
a	对称带状矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为(k+1)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量 y, 至少为(1+(n-1)*abs(incy))
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.15 ?spmv

对称压缩矩阵与向量的乘积。

$$y = \alpha * A * x + \beta * y$$

接口定义

```
void cblas_sspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *ap, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);
```

```
void cblas_dspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *ap, const double *x, const BLASINT incx, const double beta, double *y,
```

```
const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
ap	压缩数组, 规模至少是 $(n*(n+1)/2)$
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
beta	系数标量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.16 ?spr

对称压缩矩阵秩 1 更新。

$$A = \alpha * x * x' + A$$

接口定义

```
void cblas_sspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *x, const BLASINT incx, float *ap);
```

```
void cblas_dspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *x, const BLASINT incx, double *ap);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或

n	下三角部分在压缩数组 ap 中提供
alpha	矩阵 A 的阶数，大于等于 0
x	系数标量
incx	向量 x，至少为 $(1+(n-1)*abs(incx))$
ap	向量 x 的增量
	压缩数组，规模至少是 $(n*(n+1)/2)$

输出参数

ap	更新后的数组 ap
----	-----------

4.2.17 ?spr2

对称压缩矩阵秩 2 更新。

$$A = \alpha * x * y' + \alpha * y * x' + A$$

接口定义

```
void cblas_sspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *x, const BLASINT incx, const float *y, const BLASINT incy, float *a);
```

```
void cblas_dspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *x, const BLASINT incx, const double *y, const BLASINT incy, double *a);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower，则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
n	矩阵 A 的阶数，大于等于 0
alpha	系数标量
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
y	向量 y，至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 的增量
ap	压缩数组，规模至少是 $(n*(n+1)/2)$

输出参数

ap	更新后的数组 ap
----	-----------

4.2.18 ?symv

对称矩阵与向量的乘积。

$$y = \alpha * A * x + \beta * y$$

接口定义

```
void cblas_ssymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);
```

```
void cblas_dsymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx, const double beta, double *y, const BLASINT incy);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
a	对称矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为 max(1,n)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量
beta	系数标量
y	向量 y, 至少为(1+(n-1)*abs(incy))
incy	向量 y 的增量

输出参数

y	更新后的向量 y
---	----------

4.2.19 ?syr

对称矩阵秩 1 更新。

$$A = \alpha * x * x' + A$$

接口定义

```
void cblas_ssyrr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *x, const BLASINT incx, float *a, const BLASINT lda);
```

```
void cblas_dsyr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *x, const BLASINT incx, double *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
a	对称矩阵 A, 规模是 $lda*n$
lda	矩阵 A 的主维度, lda 值必须至少为 $\max(1,n)$

输出参数

a	更新后的上三角/下三角矩阵 A
---	-----------------

4.2.20 ?syr2

对称矩阵秩 2 更新。

$$A = \alpha * x * y' + \alpha * y * x' + A$$

接口定义

```
void cblas_ssyrr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const float alpha, const float *x, const BLASINT incx, const float *y, const BLASINT incy, float *a, const BLASINT lda);
```

```
void cblas_dsyr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const BLASINT n, const double alpha, const double *x, const BLASINT incx, const double *y, const BLASINT incy, double *a, const BLASINT lda);
```

输入参数

order	指定二维数组存储是按行主序
-------	---------------

	(CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用 矩阵 A 的上三角/下三角。
n	矩阵 A 的阶数, 大于等于 0
alpha	系数标量
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量
y	向量 y, 至少为 $(1+(n-1)*abs(incy))$
incy	向量 y 的增量
a	对称矩阵 A, 规模是 $lda*n$
lda	矩阵 A 的主维度, lda 值必须至少为 $\max(1,n)$
输出参数	
a	更新后的上三角/下三角矩阵 A

4.2.21 ?tbmv

三角带状矩阵与向量的乘积。

① $x=A*x$

② $x=A'*x$

③ $x=conjg(A')*x$

接口定义

```
void cblas_stbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const float *a, const blasint lda, float *x, const blasint incx);
```

```
void cblas_dtbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const double *a, const blasint lda, double *x, const blasint incx);
```

```
void cblas_ctbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const void *a, const blasint lda, void *x, const blasint incx);
```

```
void cblas_ztbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const void *a, const blasint lda, void *x, const blasint incx);
```

CBLAS_DIAG diag, const blasint n, const blasint k, const void* a, const blasint lda, void *x, const blasint incx);

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
n	矩阵 A 的阶数, 大于等于 0
k	矩阵 A 的超对角阶数, 大于等于 0
a	三角带状矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为(1+k)
alpha	系数标量
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量

输出参数

x	更新后的向量 x
---	----------

4.2.22 ?tbsv

求解系数为三角带状矩阵的线性方程组。

① $A \cdot x = b$

② $A^T \cdot x = b$

③ $\text{conjg}(A) \cdot x = b$

接口定义

```
void cblas_stbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const float *a, const blasint lda, float *x, const blasint incx);
```

```
void cblas_dtbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum
```


CBLAS_DIAG diag, const blasint n, const blasint k, const double *a, const blasint lda, double *x, const blasint incx);

void cblas_ctbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const void *a, const blasint lda, void *x, const blasint incx);

void cblas_ztbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const blasint n, const blasint k, const void* a, const blasint lda, void *x, const blasint incx);

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
n	矩阵 A 的阶数, 大于等于 0
k	矩阵 A 的超对角阶数, 大于等于 0
a	三角带状矩阵 A, 规模是 lda*n
lda	矩阵 A 的主维度, lda 值必须至少为(1+k)
alpha	系数标量
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量

输出参数

x	解向量 x
---	-------

4.2.23 ?tpmv

三角压缩矩阵与向量的乘积。

① $x = A * x$

② $x = A' * x$

③ $x = \text{conjg}(A') * x$

接口定义

```
void cblas_stpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const float *ap, float *x, const BLASINT incx);
```

```
void cblas_dtpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const double *ap, double *x, const BLASINT incx);
```

```
void cblas_ctpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *ap, void *x, const BLASINT incx);
```

```
void cblas_ztpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *ap, void *x, const BLASINT incx);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
n	矩阵 A 的阶数, 大于等于 0
ap	压缩数组, 规模至少是 $((n*(n+1))/2)$
lda	矩阵 A 的主维度, lda 值必须至少为 $(1+k)$
x	向量 x, 至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量

输出参数

x	更新后的向量 x
---	----------

4.2.24 ?tpsv

求解系数为三角压缩矩阵的线性方程组。

① $A * x = b$

② $A' * x = b$

③ $\text{conjg}(A') * x = b$

接口定义

```
void cblas_stpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const float *ap, float *x, const BLASINT incx);
```

```
void cblas_dtpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const double *ap, double *x, const BLASINT incx);
```

```
void cblas_ctpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *ap, void *x, const BLASINT incx);
```

```
void cblas_ztpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *ap, void *x, const BLASINT incx);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角。矩阵 A 的上三角部分或下三角部分在压缩数组 ap 中提供
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
n	矩阵 A 的阶数, 大于等于 0

ap	压缩数组，规模至少是 $((n*(n+1))/2)$
lda	矩阵 A 的主维度，lda 值必须至少为 $(1+k)$
x	向量 x，至少为 $(1+(n-1)*abs(incx))$
incx	向量 x 的增量

输出参数

x	解向量 x
---	-------

4.2.25 ?trmv

三角矩阵与向量的乘积。

$x=A*x$

$x=A'*x$

$x=conjg(A')*x$

接口定义

```
void cblas_strmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const BLASINT n, const float *a, const BLASINT lda, float *x, const BLASINT incx);
```

```
void cblas_dtrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const double *a, const BLASINT lda, double *x, const BLASINT incx);
```

```
void cblas_ctrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *a, const BLASINT lda, void *x, const BLASINT incx);
```

```
void cblas_ztrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *a, const BLASINT lda, void *x, const BLASINT incx);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower，则使用矩阵 A 的上三角/下三角

trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或 共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
n	矩阵 A 的阶数, 大于等于 0
a	三角矩阵 A, 规模 lda*n
lda	矩阵 A 的主维度, lda 值必须至少 max(1,n)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量
输出参数	
x	更新后的向量 x

4.2.26 ?trsv

求解系数为三角矩阵的线性方程组。

① $A * x = b$

② $A' * x = b$

③ $\text{conjg}(A') * x = b$

接口定义

```
void cblas_strsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const float *a, const BLASINT lda, float *x, const BLASINT incx);
```

```
void cblas_dtrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const double *a, const BLASINT lda, double *x, const BLASINT incx);
```

```
void cblas_ctrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const enum CBLAS_DIAG diag, const BLASINT n, const void *a, const BLASINT lda, void *x, const BLASINT incx);
```

```
void cblas_ztrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE Transa, const enum CBLAS_DIAG diag, const BLASINT n, const void *a, const BLASINT lda, void *x, const BLASINT incx);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
trans	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形。
n	矩阵 A 的阶数, 大于等于 0
a	三角矩阵 A, 规模 lda*n
lda	矩阵 A 的主维度, lda 值必须至少 max(1,n)
x	向量 x, 至少为(1+(n-1)*abs(incx))
incx	向量 x 的增量

输出参数

x	解向量 x
---	-------

4.3 BLAS-FT Level 3 函数

4.3.1 函数说明

函数族	数据类型	描述
?gemm	s、d、c、z	一般矩阵的矩阵乘法
?hemm	c、z	矩阵 A 为厄米特矩阵的矩阵乘法
?herk	c、z	厄米特矩阵 C 秩 K 更新
?her2k	c、z	厄米特矩阵 C 秩 2K 更新
?symm	s、d、c、z	矩阵 A 为对称矩阵的矩阵乘法
?syrk	s、d、c、z	对称矩阵 C 秩 K 更新
?syr2k	s、d、c、z	对称矩阵 C 秩 2K 更新
?trmm	s、d、c、z	矩阵 A 为三角矩阵的矩阵乘法
?trsm	s、d、c、z	求解三角矩阵方程

4.3.2 ?gemm

一般矩阵的矩阵乘法。

$$C = \alpha * op(A) * op(B) + \beta * C$$

接口定义

```
void cblas_sgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_TRANSPOSE transb, const BLASINT m, const BLASINT n, const BLASINT k, const float alpha, const float *a, const BLASINT lda, const float *b, const BLASINT ldb, const float beta, float *c, const BLASINT ldc);
```

```
void cblas_dgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_TRANSPOSE transb, const BLASINT m, const BLASINT n, const BLASINT k, const double alpha, const double *a, const BLASINT lda, const double *b, const BLASINT ldb, const double beta, double *c, const BLASINT ldc);
```

```
void cblas_cgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_TRANSPOSE transb, const BLASINT m, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

```
void cblas_zgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_TRANSPOSE transb, const BLASINT m, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
transa	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
transb	矩阵 B 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
m	矩阵 A、C 的行数，大于等于 0
n	矩阵 B、C 的列数，大于等于 0
k	矩阵 A 的列数，矩阵 B 的行数，大于等于 0
alpha	系数标量
a	一般矩阵 A，列存/行存，规模 lda*k/lda*m
lda	矩阵 A 的主维度，矩阵 A 为列存，

	transa=CblasNoTrans, lda 至少为 $\max(1,m)$, 否则至少为 $\max(1,k)$ 。矩阵 A 为行存, transa=CblasNoTrans, lda 至少为 $\max(1,k)$, 否则至少为 $\max(1,m)$
b	一般矩阵 B, 列存/行存, 规模 $ldb*n/ldb*k$ 矩阵 B 的主维度, 矩阵 B 为列存, transa=CblasNoTrans, ldb 至少为 $\max(1,k)$, 否则至少为 $\max(1,n)$ 。矩阵 B 为行存, transa=CblasNoTrans, ldb 至少为 $\max(1,n)$, 否则至少为 $\max(1,k)$
ldb	
beta	系数标量
c	一般矩阵 C, 列存/行存, 规模 $ldc*n/lda*m$
ldc	矩阵 C 的主维度, 矩阵为列存, ldc 至少 $\max(1,m)$, 否则 $\max(1,n)$
输出参数	
c	更新后的矩阵 C

4.3.3 ?hemm

矩阵 A 为厄米特矩阵的矩阵乘法。

① $C = \alpha * A * B + \beta * C$

② $C = \alpha * B * A + \beta * C$

接口定义

```
void cblas_chemm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

```
void cblas_zhemm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
-------	--

side	当 side=CblasLeft/CblasRight 为情况①/②
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
m	C 的行数, 大于等于 0
n	C 的列数, 大于等于 0
alpha	系数标量
a	厄米特矩阵 A, 当 side=CblasLeft/CblasRight, 规模 lda*m/lda*n
lda	矩阵 A 的主维度, 若 side=CblasLeft 则 lda 至少为 max(1,m), 若 side=CblasRight 则 lda 至少为 max(1,n)
b	一般矩阵 B, 列存/行存, 规模 ldb*n/ldb*m
ldb	矩阵 B 的主维度, 矩阵 B 为列存, ldb 至少为 max(1,m), 否则 max(1,n)
beta	系数标量
c	一般矩阵 C, 列存/行存, 规模 ldc*n/ldc*m
ldc	矩阵 C 的主维度, 矩阵为列存, ldc 至少 max(1,m), 否则 max(1,n)
输出参数	
c	更新后的矩阵 C

4.3.4 ?herk

厄米特矩阵 C 秩 K 更新。

① $C = \alpha * A * A^H + \beta * C$

② $C = \alpha * A^H * A + \beta * C$

接口定义

```
void cblas_cherk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const float alpha, const void *a, const BLASINT lda, const float beta, void *c, const BLASINT ldc);
```

```
void cblas_zherk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const double alpha, const void *a, const BLASINT lda, const double beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 C 的上三角/下三角
trans	当 trans=CblasNoTrans/CblasConjTrans 为情况①/②
n	C 的阶数, 大于等于 0
k	当 trans=CblasNoTrans/CblasConjTrans,k 为 A 的列数/行数, 大于等于 0
alpha	系数标量
a	一般矩阵 A, 当矩阵为列存时, trans=CblasNoTrans/CblasConjTrans, 规模 lda*k/lda*n, 当矩阵为行存时, trans=CblasNoTrans/CblasConjTrans, 规模 lda*n/lda*k
lda	矩阵 A 的主维度, 矩阵为列存, trans=CblasNoTrans, lda 至少 max(1, n), 否则至少 max(1,k)。矩阵为行存, trans=CblasNoTrans, lda 至少 max(1,k), 否则至少 max(1,n)
beta	系数标量
c	厄米特矩阵 C, 规模 ldc*n
ldc	矩阵 C 的主维度, ldc 至少 max(1,n)
输出参数	
c	更新后的上三角/下三角矩阵 C

4.3.5 ?her2k

厄米特矩阵 C 秩 2K 更新。

① $C = \alpha * A * B^H + \text{conjg}(\alpha) * B * A^H + \beta * C$

② $C = \alpha * A^H * B + \text{conjg}(\alpha) * B^H * A + \beta * C$

接口定义

```
void cblas_?her2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const float beta, void *c, const BLASINT ldc);
```

```
void cblas_zher2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const double beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 C 的上三角/下三角
trans	当 trans=CblasNoTrans/CblasConjTrans 为情况①/②
n	C 的阶数, 大于等于 0
k	当 trans=CblasNoTrans/CblasConjTrans, k 为 A 的列数/行数, 大于等于 0
alpha	系数标量
a	一般矩阵 A, 当矩阵为列存时, trans=CblasNoTrans/CblasConjTrans, 规模 lda*k/lda*n, 当矩阵为行存时, trans=CblasNoTrans/CblasConjTrans, 规模 lda*n/lda*k
lda	矩阵 A 的主维度, 矩阵为列存, trans=CblasNoTrans, lda 至少 max(1, n), 否则至少 max(1, k)。矩阵为行存, trans=CblasNoTrans, lda 至少 max(1, k), 否则至少 max(1, n)
b	一般矩阵 B, 当矩阵为列存时, trans=CblasNoTrans/CblasConjTrans, 规模 ldb*k/ldb*n, 当矩阵为行存时, trans=CblasNoTrans/CblasConjTrans, 规模 ldb*n/ldb*k
ldb	矩阵 B 的主维度, 矩阵为列存, trans=CblasNoTrans, lda 至少 max(1, n), 否则至少 max(1, k)。矩阵为行存, trans=CblasNoTrans, lda 至少 max(1, k), 否则至少 max(1, n)
beta	系数标量

c	厄米特矩阵 C，规模 ldc*n
ldc	矩阵 C 的主维度，矩阵为列存，ldc 至少 max(1,n)
输出参数	
c	更新后的上三角/下三角矩阵 C

4.3.6 ?symm

矩阵 A 为对称矩阵的矩阵乘法。

① $C = \alpha * A * B + \beta * C$

② $C = \alpha * B * A + \beta * C$

接口定义

```
void cblas_ssymm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const float alpha, const float *a, const BLASINT lda, const float *b, const BLASINT ldb, const float beta, float *c, const BLASINT ldc);
```

```
void cblas_dsymm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const double alpha, const double *a, const BLASINT lda, const double *b, const BLASINT ldb, const double beta, double *c, const BLASINT ldc);
```

```
void cblas_csymm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

```
void cblas_zsymm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
side	当 side=CblasLeft/CblasRight 为情况①/②
uplo	如果 uplo=CblasUpper/CblasLower，则使用矩

	阵 A 的上三角/下三角
m	C 的行数, 大于等于 0
n	C 的列数, 大于等于 0
alpha	系数标量
a	对称矩阵 A, 当 side=CblasLeft/CblasRight, 规模 $lda*m/lda*n$
lda	矩阵 A 的主维度, 若 side=CblasLeft 则 lda 至少为 $\max(1,m)$, 若 side=CblasRight 则 lda 至少为 $\max(1,n)$
b	一般矩阵 B, 列存/行存, 规模 $ldb*n/ldb*m$
ldb	矩阵 B 的主维度, 矩阵 B 为列存, ldb 至少为 $\max(1,m)$, 否则 $\max(1,n)$
beta	系数标量
c	一般矩阵 C, 列存/行存, 规模 $ldc*n/ldc*m$
ldc	矩阵 C 的主维度, 矩阵为列存, ldc 至少 $\max(1,m)$, 否则 $\max(1,n)$
输出参数	
c	更新后的矩阵 C

4.3.7 ?syrk

对称矩阵 C 秩 K 更新。

① $C = \alpha * A * A' + \beta * C$

② $C = \alpha * A' * A + \beta * C$

接口定义

```
void cblas_ssyrrk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const float alpha, const float *a, const BLASINT lda, const float beta, float *c, const BLASINT ldc);
```

```
void cblas_dsyrk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const double alpha, const double *a, const BLASINT lda, const double beta, double *c, const BLASINT ldc);
```

```
void cblas_csyrk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const
```

```
BLASINT lda, const void *beta, void *c, const BLASINT ldc);
```

```
void cblas_zsyrk(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 C 的上三角/下三角
trans	当 trans=CblasNoTrans/CblasTrans/CblasConjTrans 为情况①/②/②
n	C 的阶数, 大于等于 0
k	当 trans=CblasNoTrans/CblasTrans/CblasConjTrans, k 为 A 的列数/行数/行数, 大于等于 0
alpha	系数标量
a	一般矩阵 A, 当矩阵为列存时, trans=CblasNoTrans/CblasTrans/CblasConjTrans, 规模 lda*k/lda*n/lda*n, 当矩阵为行存时, trans=CblasNoTrans/CblasTrans/CblasConjTrans, 规模 lda*n/lda*k/lda*k
lda	矩阵 A 的主维度, 矩阵为列存, trans=CblasNoTrans, lda 至少 max(1,n), 否则至少 max(1,k)。矩阵为行存, trans=CblasNoTrans, lda 至少 max(1,k), 否则至少 max(1,n)
beta	系数标量
c	对称矩阵 C, 规模 ldc*n
ldc	矩阵 C 的主维度, ldc 至少 max(1,n)

输出参数

c	更新后的上三角/下三角矩阵 C
---	-----------------

4.3.8 ?syr2k

对称矩阵 C 秩 2K 更新。

① $C = \alpha * A * B' + \alpha * B * A' + \beta * C$

② $C = \alpha * A' * B + \alpha * B' * A + \beta * C$

接口定义

```
void cblas_ssy2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const float alpha, const float *a, const BLASINT lda, const float *b, const BLASINT ldb, const float beta, float *c, const BLASINT ldc);
```

```
void cblas_dsy2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const double alpha, const double *a, const BLASINT lda, const double *b, const BLASINT ldb, const double beta, double *c, const BLASINT ldc);
```

```
void cblas_csy2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

```
void cblas_zsy2k(const enum CBLAS_ORDER order, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE trans, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 C 的上三角/下三角
trans	当 trans=CblasNoTrans/CblasTrans/CblasConjTrans 为情况①/②/②
n	C 的阶数, 大于等于 0
k	当 trans=CblasNoTrans/CblasTrans/CblasConjTrans,k 为 A 的列数/行数/行数, 大于等于 0
alpha	系数标量

a	一般矩阵 A，当矩阵为列存时， trans=CblasNoTrans/CblasTrans/CblasConjTrans， 规模 $lda*k/lda*n/lda*n$ ，当矩阵为行存时， trans=CblasNoTrans/CblasTrans/CblasConjTrans， 规模 $lda*n/lda*k/lda*k$
lda	矩阵 A 的主维度，矩阵为列存，trans=CblasNoTrans，lda 至少 $\max(1, n)$ ，否则至少 $\max(1, k)$ 。矩阵为行存， trans=CblasNoTrans，lda 至少 $\max(1, k)$ ，否则至少 $\max(1, n)$
b	一般矩阵 B，当矩阵为列存时， trans=CblasNoTrans/CblasTrans/CblasConjTrans， 规模 $ldb*k/ldb*n/ldb*n$ ，当矩阵为行存时， trans=CblasNoTrans/CblasTrans/CblasConjTrans， 规模 $ldb*n/ldb*k/ldb*k$
ldb	矩阵 B 的主维度，矩阵为列存，trans=CblasNoTrans，lda 至少 $\max(1, n)$ ，否则至少 $\max(1, k)$ 。矩阵为行存， trans=CblasNoTrans，lda 至少 $\max(1, k)$ ，否则至少 $\max(1, n)$
beta	系数标量
c	厄米特矩阵 C，规模 $ldc*n$
ldc	矩阵 C 的主维度，矩阵为列存，ldc 至少 $\max(1, n)$

输出参数

c	更新后的上三角/下三角矩阵 C
---	-----------------

4.3.9 ?trmm

矩阵 A 为三角矩阵的矩阵乘法。

① $B = \alpha * op(A) * B$

$B = \alpha * B * op(A)$

接口定义

```
void cblas_strmm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const BLASINT m, const BLASINT n, const float alpha, const float *a, const BLASINT lda, float *b, const BLASINT ldb);
```

```
void cblas_dtrmm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum
```



```
CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const
BLASINT m, const BLASINT n, const double alpha, const double *a, const
BLASINT lda, double *b, const BLASINT ldb);
```

```
void cblas_ctrmm(const enum CBLAS_ORDER order, const enum
CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum
CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const
BLASINT m, const BLASINT n, const void *alpha, const void *a, const
BLASINT lda, void *b, const BLASINT ldb);
```

```
void cblas_ztrmm(const enum CBLAS_ORDER order, const enum
CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum
CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const
BLASINT m, const BLASINT n, const void *alpha, const void *a, const
BLASINT lda, void *b, const BLASINT ldb);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
side	当 side=CblasLeft/CblasRight 为情况①/②
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角
transa	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或共轭矩阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit, 则矩阵 A 为单元/非单元三角形
m	矩阵 B 的行数, 大于等于 0
n	矩阵 B 的列数, 大于等于 0
alpha	参数标量
a	三角矩阵 A, 对称矩阵 A, 当 side=CblasLeft/CblasRight, 规模 lda*m/lda*n
lda	矩阵 A 的主维度, 若 side=CblasLeft 则 lda 至少为 max(1,m), 若 side=CblasRight 则 lda 至少为 max(1,n)
b	一般矩阵 B, 列存/行存, 规模 ldb*n/ldb*m
ldb	矩阵 B 的主维度, 矩阵 B 为列存, ldb 至少为 max(1,m), 否则 max(1,n)

输出参数

b

更新后的矩阵 B

4.3.10 ?trsm

求解三角矩阵方程。

① $\text{op}(A) * X = \alpha * B$

② $X * \text{op}(A) = \alpha * B$

接口定义

```
void cblas_strsm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const BLASINT m, const BLASINT n, const float alpha, const float *a, const BLASINT lda, float *b, const BLASINT ldb);
```

```
void cblas_dtrsm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const BLASINT m, const BLASINT n, const double alpha, const double *a, const BLASINT lda, double *b, const BLASINT ldb);
```

```
void cblas_ctrsm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_DIAG diag, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, void *b, const BLASINT ldb);
```

```
void cblas_ztrsm(const enum CBLAS_ORDER order, const enum CBLAS_SIDE side, const enum CBLAS_UPLO uplo, const enum CBLAS_TRANSPOSE transa, const enum CBLAS_DIAG diag, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, void *b, const BLASINT ldb);
```

输入参数

order	指定二维数组存储是按行主序 (CblasRowMajor) 还是按列主序 (CblasColMajor)
side	当 side=CblasLeft/CblasRight 为情况①/②
uplo	如果 uplo=CblasUpper/CblasLower, 则使用矩阵 A 的上三角/下三角

transa	矩阵 A 为常规矩阵 (CblasNoTrans)、转置矩阵 (CblasTrans) 或 共 轭 矩 阵 (CblasConjTrans)
diag	如果 diag=CblasUnit/CblasNonUnit,则矩阵 A 为单元/非单元三角形
m	矩阵 B 的行数, 大于等于 0
n	矩阵 B 的列数, 大于等于 0
alpha	参数标量
a	三 角 矩 阵 A , 对 称 矩 阵 A , 当 side=CblasLeft/CblasRight,规模 lda*m/lda*n
lda	矩阵 A 的主维度,若 side=CblasLeft 则 lda 至少为 max(1,m), 若 side=CblasRight 则 lda 至少为 max(1,n)
b	一般矩阵 B, 列存/行存, 规模 ldb*n/ldb*m
ldb	矩阵 B 的主维度, 矩阵 B 为列存, ldb 至少为 max(1,m), 否则 max(1,n)
输出参数	
b	解矩阵 X.